

Evo Aura Existing Files - Alteration Guide to 41 Python Files

Purpose: give Codex exact instructions to alter the uploaded existing files without excluding any current file, while moving the app into a clean 41-file PyQt6 architecture. Target OS is Windows 10 and Windows 11 only. Do not downgrade or change code for Windows 7/8 compatibility.

Main rule: migrate safely, keep the app runnable after each step, and do not delete original code until the new file passes import and page-load tests.

1. Current existing file inventory

Existing file	Lines	Size KB	Qt	Main alteration action
evoaura_app(6).py	1960	87.5	PyQt6	Split into main.py, core/db.py, core/auth_service.py, core/ui_helpers.py, core/app_shell.py, auth pages, dashboard page.
loyalty_page.py	691	28.0	PyQt6	Move to pages/customers/loyalty_page.py; split loyalty rules/service/repository.
credit_management_page.py	634	27.7	PyQt6	Move to pages/customers/due_collection_page.py; split payment/reminder dialogs and service/repository.
sidebar(2).py	1052	44.2	PyQt6	Move to core/sidebar.py. Keep one PyQt6 sidebar only. Make NAV_ITEMS the source for routes.
customer_page.py	870	40.9	PyQt6	Move to pages/customers/customer_center_page.py and split DB queries to customer repository/service.
low_stock_page.py	589	25.6	PyQt6	Move to pages/inventory/reorder_control_page.py; split reorder analytics/service/repository.
purchase_orders_page.py	731	33.8	PyQt6	Move to pages/inventory/purchase_invoices_page.py; split dialog/service/repository.
dashboard(1).py	1486	60.9	PyQt5	PyQt5 legacy dashboard. Convert useful dashboard widgets/stats to PyQt6 dashboard_page.py or archive after migration.
supplier_page(1).py	1801	78.7	PyQt6	Split into supplier page, supplier form/detail/list components, supplier service, supplier repository.
product_page(7).py	7269	329.2	PyQt6	Split immediately. Keep ProductPage as controller; move DB queries to repository and business rules to service.
app_branding_qt5.py	21	0.5	PyQt5	Archive only. Do not use in final Windows 10/11 PyQt6 build.
app_branding.py	21	0.5	PyQt6	Move to core/app_branding.py. Use this PyQt6 file in final app.
input_behavior_qt5.py	57	1.9	PyQt5	Archive only. Do not use in final Windows 10/11 PyQt6 build.
input_behavior.py	82	2.5	PyQt6	Move to core/input_behavior.py. Use this PyQt6 guard globally.

2. Diagnosis from the existing files

- product_page(7).py has 7,269 lines. It should be split first because it contains UI, SQLite queries, validation, dialogs, tables, stock logic, variants, supplier links, history, and audit in one file.
- evoaura_app(6).py has 1,960 lines. It contains boot flow, DB layer, auth service, UI helpers, auth pages, AppShell, MainWindow, and dashboard. It should become multiple core/auth files.
- sidebar(2).py is PyQt6 and should become the final sidebar. It already contains the 23 main route keys.
- dashboard(1).py is PyQt5 while the main app is PyQt6. Do not mix PyQt5 and PyQt6 in the final EXE. Convert only useful dashboard logic to PyQt6 or archive the Qt5 file.
- app_branding.py and input_behavior.py are the PyQt6 versions and should be used in the final app.

- app_branding_qt5.py and input_behavior_qt5.py are PyQt5 helpers. Keep them only in archive/reference; do not use them in final Windows 10/11 PyQt6 build.
- Several pages import C, FIELD_SS, _NO_ARROW, _apply_combo_delegate, and init_product_table from product_page.py. During migration, move these to core/theme.py, core/ui_helpers.py, and core/migrations.py, then update imports gradually.

3. Exact alteration for each existing file

evoaura_app(6).py

Current: 1960 lines, PyQt6. Important classes: DB, Toast, SI, GB, RightPanel, LV, V_AskDB, V_MasterAuth, V_CompanySettings, V_Signup, V_QR, V_Login.... Important functions: hash_pw, gen_recovery, totp_uri_pixmap, verify_master, verify_admin_totp, F, L, gap, hline, card_style, __init__, _migrate, company_exists, save_company....

Alteration: Split into main.py, core/db.py, core/auth_service.py, core/ui_helpers.py, core/app_shell.py, auth pages, dashboard page.

Existing class/function group	Move to target
DB class	core/db.py, with database path in AppData/Local/EvoAura/databases.
hash_pw, gen_recovery, TOTP helpers	core/auth_service.py.
F, L, gap, hline, card_style, Toast, SI, GB	core/ui_helpers.py.
C and APP_SS	core/theme.py.
V_AskDB, V_MasterAuth, V_CompanySettings, V_Signup, V_QR, V_Login, V_OTP	pages/auth/*.py.
AppShell, MainWindow, route handling	core/app_shell.py + core/routes.py.
_DashboardPage	pages/dashboard_page.py.

loyalty_page.py

Current: 691 lines, PyQt6. Important classes: LoyaltyPointsDialog, LoyaltySettingsDialog, LoyaltyPage. Important functions: _money, _button, _card, _table, _rows, _table_columns, init_loyalty_tables, tier_for_points, loyalty_rows, loyalty_kpis, __init__, _selected_customer, _update_hint, _save....

Alteration: Move to pages/customers/loyalty_page.py; split loyalty rules/service/repository.

credit_management_page.py

Current: 634 lines, PyQt6. Important classes: PaymentDialog, CreditManagementPage. Important functions: _money, _button, _card, _table, _rows, init_credit_tables, _parse_date, outstanding_bills, credit_rows, credit_kpis, next_receipt, __init__, _save, __init__....

Alteration: Move to pages/customers/due_collection_page.py; split payment/reminder dialogs and service/repository.

sidebar(2).py

Current: 1052 lines, PyQt6. Important classes: _NavButton, _FlyoutRow, _FlyoutMenu, _NavGroup, Sidebar. Important functions: _load_icon, _pixmap, _divider, __init__, set_active, set_expanded, _refresh, paintEvent, enterEvent, leaveEvent, mousePressEvent, __init__, _apply_style, enterEvent....

Alteration: Move to core/sidebar.py. Keep one PyQt6 sidebar only. Make NAV_ITEMS the source for routes.

customer_page.py

Current: 870 lines, PyQt6. Important classes: CustomerFormDialog, CustomerPage. Important functions: `_money`, `_button`, `_card`, `_table`, `init_customer_tables`, `_rows`, `_table_columns`, `get_customer_invoices`, `get_purchased_products`, `customer_rows`, `customer_kpis`, `next_customer_code`, `__init__`, `_load`....

Alteration: Move to `pages/customers/customer_center_page.py` and split DB queries to `customer repository/service`.

low_stock_page.py

Current: 589 lines, PyQt6. Important classes: LowStockPage. Important functions: `_money`, `_button`, `_card`, `_table`, `init_low_stock_tables`, `_sales_for_period`, `get_reorder_rows`, `get_reorder_kpis`, `match`, `__init__`, `_reload_filter_options`, `refresh`, `_cell_clicked`, `_show_drawer`....

Alteration: Move to `pages/inventory/reorder_control_page.py`; split `reorder analytics/service/repository`.

purchase_orders_page.py

Current: 731 lines, PyQt6. Important classes: PurchaseInvoiceDialog, PurchaseOrdersPage. Important functions: `_money`, `_button`, `_card`, `_table`, `init_purchase_tables`, `_rows`, `purchase_rows`, `purchase_kpis`, `__init__`, `_apply_prefill`, `_supplier_changed`, `_spin`, `_add_line`, `_remove_line`....

Alteration: Move to `pages/inventory/purchase_invoices_page.py`; split `dialog/service/repository`.

dashboard(1).py

Current: 1486 lines, PyQt5. Important classes: KpiCard, SparkBar, HomePage, ComingSoonPage, `_NavButton`, `_FlyoutMenu`, `NavGroupWidget`, `Sidebar`, `TopBar`, `Dashboard`. Important functions: `load_company_info`, `pixmap_from_blob`, `_q`, `load_dashboard_stats`, `_lbl`, `_divider`, `_clear_layout`, `safe`, `__init__`, `__init__`, `paintEvent`, `__init__`, `_build_banner`, `_tick_clock`....

Alteration: PyQt5 legacy dashboard. Convert useful dashboard widgets/stats to PyQt6 `dashboard_page.py` or archive after migration.

supplier_page(1).py

Current: 1801 lines, PyQt6. Important classes: InvoiceDetailDialog, ProductDetailDialog, SupplierFormWidget, SupplierDetailWidget, SupplierListWidget, SupplierPage. Important functions: `_rows`, `_row`, `_money`, `_number`, `_gst_percent`, `init_supplier_tables`, `_supplier_product_rows`, `_variant_labels`, `_purchase_log_rows`, `get_supplier_inventory`, `get_supplier_purchases`, `get_invoice_items`, `get_supplier_summary`, `get_suppliers`....

Alteration: Split into supplier page, supplier form/detail/list components, supplier service, supplier repository.

product_page(7).py

Current: 7269 lines, PyQt6. Important classes: `_NoWheelValueFilter`, `_ComboDelegate`, `ToggleSwitch`, `_MonthlySalesChart`, `_GBtn`, `StockAdjDialog`, `PurchaseStockDialog`, `ManualStockDialog`, `PurchaseInvoiceDetailDialog`, `_SlidePanel`, `ProductFormWidget`, `ProductListWidget`.... Important functions: `_apply_combo_delegate`, `init_product_table`, `get_all_products`, `get_product_admin_kpis`, `get_categories`, `get_all_supplier_names`, `get_supplier_by_name`, `create_supplier`, `save_product_supplier`, `get_product_suppliers`, `get_saved_sub_categories`, `get_next_item_code`, `_ean13_checksum`, `get_next_ean13`....

Alteration: Split immediately. Keep `ProductPage` as controller; move DB queries to repository and business rules to service.

Split plan for product_page(7).py:

Target module	Move these responsibilities
pages/inventory/product_page.py	ProductPage controller only: layout, open add/edit overlay, page refresh.
pages/inventory/product_list_widget.py	ProductListWidget, filters, KPI chips, table actions, preview drawer.
pages/inventory/product_form_widget.py	ProductFormWidget, 7 tabs, form load/save, deferred build.
pages/inventory/product_dialogs.py	StockAdjDialog, PurchaseStockDialog, ManualStockDialog, PurchaseInvoiceDetailDialog.
services/product_service.py	Validation, item code/barcode generation, margin/GST calculations, save/update orchestration.
repositories/product_repository.py	get_all_products, get_product_full, price history, sales rows, variants, suppliers, stock adjustment SQL.
models/product.py	Product dataclass/dict normalization and default values.
core/theme.py / core/ui_helpers.py	C, FIELD_SS, TAB_SS, LABEL_SS, combo delegate, reusable field/table helpers.
core/migrations.py	init_product_table and all product-related table/index migrations.

app_branding_qt5.py

Current: 21 lines, PyQt5. Important classes: none detected. Important functions: apply_app_icon_qt5.

Alteration: Archive only. Do not use in final Windows 10/11 PyQt6 build.

app_branding.py

Current: 21 lines, PyQt6. Important classes: none detected. Important functions: apply_app_icon.

Alteration: Move to core/app_branding.py. Use this PyQt6 file in final app.

input_behavior_qt5.py

Current: 57 lines, PyQt5. Important classes: InputBehaviorGuardQt5. Important functions: ensure_global_input_guard_qt5, eventFilter.

Alteration: Archive only. Do not use in final Windows 10/11 PyQt6 build.

input_behavior.py

Current: 82 lines, PyQt6. Important classes: InputBehaviorGuard. Important functions: ensure_global_input_guard, _spin_for, _value_control_for, _prepare_spin, eventFilter.

Alteration: Move to core/input_behavior.py. Use this PyQt6 guard globally.

4. Migration procedure for Codex

Step 1 - Backup and normalize names

Copy all current files to a backup folder. Rename working copies to clean names only after imports are updated: `evoaura_app.py`, `sidebar.py`, `product_page.py`, `supplier_page.py`, `dashboard.py`.

Step 2 - Create folder tree

Create `core/`, `pages/auth/`, `pages/sales/`, `pages/inventory/`, `pages/customers/`, `pages/finance/`, `pages/reports/`, `pages/settings/`, `services/`, `repositories/`, `models/`, `assets/icons/`, `data/`.

Step 3 - Extract core first

Move `C/APP_SS` to `core/theme.py`, helpers to `core/ui_helpers.py`, DB to `core/db.py`, migrations to `core/migrations.py`, auth to `core/auth_service.py`.

Step 4 - Extract auth pages

Move all `V_*` auth classes from `evoaura_app(6).py` into `pages/auth` files. Keep signal names and behavior unchanged.

Step 5 - Move app shell

Move `MainWindow/AppShell` route logic into `core/app_shell.py`. `main.py` should only start `QApplication` and open `MainWindow`.

Step 6 - Move sidebar

Move `sidebar(2).py` to `core/sidebar.py`. Keep `NAV_ITEMS` and active route keys unchanged.

Step 7 - Split product page

Split `product_page(7).py` as listed. First move reusable `C/FIELD_SS` out; then update all imports in customer, credit, loyalty, low stock, purchase, and supplier pages.

Step 8 - Move existing business pages

Move `customer_page.py`, `credit_management_page.py`, `loyalty_page.py`, `purchase_orders_page.py`, `low_stock_page.py`, `supplier_page(1).py` into their target pages folders. Then move their SQL functions to `repositories` and analytics to `services`.

Step 9 - Add missing pages

Create real or placeholder pages for sales, finance, reports, settings pages not yet implemented. Every sidebar route must resolve to a page.

Step 10 - Keep temporary wrappers

During migration only, root files can re-export classes from new locations so old imports still work. Remove wrappers after all imports are updated.

Step 11 - Run imports and UI smoke tests

Run `python -m py_compile` on all `.py` files, then open app, create DB, login, click every sidebar page, add/edit a product, add supplier, create purchase invoice, check low-stock, customer, due, loyalty.

5. Temporary wrapper examples

Use wrappers only during migration to avoid breaking existing imports while files are being moved.

```
# product_page.py - temporary wrapper
from pages.inventory.product_page import ProductPage
from pages.inventory.product_form_widget import ProductFormWidget
```

```
from pages.inventory.product_list_widget import ProductListWidget
from pages.inventory.product_dialogs import PurchaseStockDialog
from core.theme import C, FIELD_SS, TAB_SS, LABEL_SS, SEC_HDR_SS
from core.ui_helpers import _NO_ARROW, _apply_combo_delegate
from core.migrations import init_product_table
```

```
# sidebar.py - temporary wrapper
from core.sidebar import Sidebar, NAV_ITEMS
```

6. Performance rules for smooth app

- Use lazy page loading in AppShell: build a page only when the sidebar route is clicked.
- Keep product/customer/supplier tables paginated. Do not SELECT all rows for large tables.
- For very large tables, prefer QTableView + QAbstractTableModel instead of QTableWidget.
- Add database indexes for barcode, SKU, item_code, customer phone, invoice number, invoice date, supplier code.
- Run export, import, backup, and large report generation in QThread/QRunnable; do not block the main UI thread.
- Cache icons and avoid reloading/tinting PNGs on every hover event.
- Do not apply huge setStyleSheet calls repeatedly in refresh loops. Apply app style once and update only needed widgets.

7. Windows 10/11 EXE rule

Final build target is Windows 10 and later. Do not add old Windows 7/8 workarounds. Store database files under the user profile, not Program Files. Recommended location: %LOCALAPPDATA%/EvoAura/databases/<company>.db. The installer may install the EXE in Program Files, but data must be created in AppData.

```
Recommended PyInstaller command:
pyinstaller --noconfirm --windowed --name EvoAura ^
--add-data "assets;assets" ^
--add-data "icons;icons" ^
main.py
```

8. Acceptance checklist

- App starts from main.py.
- New computer / no DB opens Ask DB page.
- Company DB is created in AppData/Local/EvoAura/databases.
- Company setup, admin signup, QR, login, OTP all work.
- All 23 sidebar routes open without import errors.
- Existing Product, Supplier, Purchase, Low Stock, Customer, Due Collection, and Loyalty features still work.
- No PyQt5 import remains in final runtime files.
- EXE opens without console and without lag on Windows 10/11.